

UNIVERSIDAD DE GRANADA

ETSIIT INFORMÁTICA Y TELECOMUNICACIÓN



UNIVERSIDAD
DE GRANADA

Departamento de Ciencias de la
Computación e Inteligencia Artificial

Metaheurísticas

Guión de Prácticas

Práctica 2:

Técnicas de Búsqueda basadas en
Poblaciones para el Problema de
Asignación Con Restricciones (PAR)

Curso 2025-26

Tercero en Grado en Ingeniería Informática

1 OBJETIVOS

El objetivo de esta práctica es estudiar el funcionamiento de las *Técnicas de Búsqueda basadas en Poblaciones* en la resolución del Problema de Asignación Con Restricciones (PAR) descrito en las transparencias del Seminario 2, y planteados en el Seminario 3. Para ello, se requerirá que el estudiante adapte los siguientes algoritmos a dicho problema:

- Algoritmos Genéticos: Dos variantes generacionales elitistas (AGGs) y otras dos estacionarias (AGEs) descritas en el Seminario 3. Aparte del esquema de evolución, la única diferencia entre los modelos de AGGs será el operador de cruce empleado.
- Algoritmos Meméticos: Tres variantes de algoritmos meméticos (AMs) basadas en un AGG, descritas en el Seminario 3. La única diferencia entre las tres variantes de AMs serán los parámetros considerados para definir la aplicación de la búsqueda local. Para diseñar los AMs, se utilizará el método de Búsqueda Local (BL) descrito en la Práctica 1.

El estudiante deberá comparar los resultados obtenidos en la serie de casos del problema con los proporcionados por los algoritmos *Greedy*, los modelos de Algoritmo Aleatorio y la Búsqueda Local (BL) implementados en la práctica 1.

La práctica se evalúa sobre un total de **2.5 puntos**, distribuidos de la siguiente forma:

- AGGs (1 punto).
- AGEs (1 punto).
- AMs (0.5 punto).

La fecha límite de entrega será el **domingo 10 de mayo de 2026** antes de las 23:55 horas. La entrega de la práctica se realizará por internet a través del espacio de la asignatura en PRADO.

2 TRABAJO A REALIZAR

El estudiante deberá de desarrollar los distintos algoritmos al problema planteado. Los métodos desarrollados serán ejecutados sobre una serie de casos del problema. Se realizará un estudio comparativo de los resultados obtenidos y se analizará el comportamiento de cada algoritmo en base a dichos resultados. **Este análisis influirá decisivamente en la calificación final de la práctica.**

En las secciones siguientes se describen los aspectos relacionados con cada algoritmo a desarrollar y las tablas de resultados a obtener.

3 COMPONENTES DE LOS ALGORITMOS

Los algoritmos de esta práctica tienen en común las siguientes componentes:

Esquema de representación Se seguirá la representación entera basada en un vector S de tamaño n (número de instancias del conjunto de datos) con valores en $\{1, \dots, k\}$ (siendo k el número de agrupamientos) que indican el cluster asociado a cada instancia, explicado en las transparencias del seminario.

Función objetivo Al igual que en la práctica anterior, será la combinación mediante el peso λ de las medidas de desviación general de la partición y el número de restricciones incumplidas resultantes del agrupamiento obtenido por la solución representada en el vector S :

$$fitness(S) = Distancia_{Intra}(S) + \lambda \cdot RestriccionesIncumplidas(S) \quad (1)$$

El valor de λ está explicado en las transparencias del Seminario 2. El objetivo será minimizar esta función.

Generación de la solución inicial La solución inicial se generará de forma aleatoria escogiendo un valor en $\{1, \dots, k\}$ en cada posición del vector S , comprobando que cada cluster tiene al menos una instancia asignada.

Esquema de generación de vecinos Se empleará el movimiento de cambio de cluster que altera el vector S sustituyendo el valor S_i en una posición i -ésima por otro valor $l \in \{1, \dots, k\}, l \neq S_i$, comprobando que no deja ningún cluster vacío.

Criterio de aceptación Se considera una mejora cuando disminuye el valor de la función objetivo.

4 ALGORITMOS GENÉTICOS

4.1 Descripción de los algoritmos

Los Algoritmos Genéticos de esta práctica presentarán las siguientes componentes:

Esquema de evolución Como se ha comentado, se considerarán dos versiones, una basada en el esquema generacional con elitismo (AGG) y otra basada en el esquema estacionario (AGE). En el primero se seleccionará una población de padres del mismo tamaño que la población genética mientras que en el segundo se seleccionarán únicamente dos padres en cada iteración.

Operador de selección Se usará el torneo, consistente en elegir aleatoriamente varios individuos de la población y seleccionar el mejor de ellos. En el esquema generacional, se aplicarán tantos torneos como individuos existan en la población genética, y se incluyen a los individuos ganadores en la nueva población. En el esquema estacionario, se aplicará dos veces el torneo para elegir los dos padres que serán posteriormente recombinados (cruzados).

Esquema de reemplazo En el esquema generacional, la población de hijos sustituye automáticamente a la actual. Para evitar perder la mejor solución encontrada, se comprueba si la mejor solución de la nueva población es peor que la mejor de la población anterior. Si es el caso, se reemplaza la peor solución de la nueva por la mejor de la antigua. En el estacionario, los dos descendientes generados tras el cruce y la mutación sustituyen a los dos peores de la población actual, en caso de ser mejores que ellos.

Operador de cruce Se emplearán dos operadores de cruce para representación real, explicados en las transparencias de teoría del tema de AGs y del Seminario 3. Uno de ellos será el operador uniforme mientras que el otro será el cruce por segmento fijo. **Esto resultará en el desarrollo de cuatro AGs distintos, dos generacionales (AGG-UN y AGG-SF) y dos estacionarios (AGE-UN y AGE-SF).** Se recuerda someramente los operadores:

- **Operador Uniforme:** Dado dos soluciones S^1, S^2 se define el operador uniforme como aquel:

$$Off_i^1, Off_i^2 = \begin{cases} S_i^1, S_i^2 & \text{if } rand() < 0,5 \\ S_i^2, S_i^1 & \text{otherwise} \end{cases} \quad (2)$$

Es decir, se intercambian aleatoriamente para cada posición del descendiente el valor de uno de las soluciones *padre* (y el otro descendiente con el valor de la otra).

- **Operador de Segmento Fijo:** En este operador, dadas las soluciones S^1, S^2 se calculan los valores de los descendientes Off^1, Off^2 de la siguiente forma:

1. Se genera una posición pos como el inicio de un segmento de tamaño FN .
2. Se calcula $min_{pos} = pos$, $final = pos + FN$ y $max_{pos} = \min(N - 1, final)$. Donde N es el tamaño de la solución, y FN el tamaño del vector. **final** es la posición final del segmento, y max_{pos} la posición final posible teniendo en cuenta la longitud de la solución.
3. Si $final > N$ se calcula la posición final $final_{rem} \leftarrow final \% N$, y se intercambia el trozo entre 0 y dicha posición según Equation 3.

$$\{Off_i^1 = S_i^1, \quad Off_i^2 = S_i^2 \quad \forall i \in [0, final_{rem}] \quad (3)$$

Y en cualquier caso se intercambian los elementos en las posiciones entre min_{pos} y max_{pos} según Ecuación (4).

$$\begin{cases} Off_i^1 = S_i^1, & Off_i^2 = S_i^2 & \forall i \in [min_{pos}, max_{pos}] \\ Off_i^1 = S_i^1, & Off_i^2 = S_i^2 & \forall i \notin [min_{pos}, max_{pos}], r() \leq 0,5 \\ Off_i^1 = S_i^2, & Off_i^2 = S_i^1 & \forall i \notin [min_{pos}, max_{pos}], r() > 0,5 \end{cases} \quad (4)$$

Operador de mutación Emplearemos el operador de mutación uniforme. Supongamos que hemos determinado que debe mutar un gen en un cromosoma. Basta con generar dos números aleatorios, uno para determinar el gen que muta (en el intervalo $\{0, \dots, N-1\}$) y otro para determinar el nuevo valor (que debe ser distinto del actual y mantener las restricciones). Coincidirá por tanto con el operador de generación de vecinos de la BL.

4.2 Valores de los parámetros y ejecuciones

El tamaño de la población será de 50 cromosomas. la probabilidad de cruce será 0,8 en el AGG y 1 en el AGE (siempre se cruzan los dos padres). La probabilidad de mutación (por cromosoma) será de 0,1 en todos los casos. El criterio de parada en las dos versiones del AG consistirá en realizar 100000 evaluaciones de la función objetivo. Para el cruce de segmento fijo en cada cruce la posición inicial será aleatoria, y como tamaño será un valor aleatorio entre 1 y $N/3$.

5 ALGORITMOS MEMÉTICOS

5.1 Búsqueda Local Suave, BLS

Aquí se resume el método de Búsqueda Local usado dentro de los algoritmos meméticos, considerado como Búsqueda Local Suave, BLS, descrito en el Seminario 3.

En esta Búsqueda Local, a diferencia del algoritmo de la práctica 1, se escoge una posición a mejorar, y luego se recorre los distintos valores posibles para escoger el mejor en esa posición. Una vez encontrado se cambia al nuevo valor (si es mejor).

Los pasos, por tanto, serían los indicados en Algoritmo 1.

El algoritmo para cuando se hayan explorado un número instancias sin mejorar (fallos), se hayan explorado todas las instancias, o se llegue al número máximo de evaluaciones (igual a 100 por ejecución) o bien todas las posiciones han sido exploradas.

5.2 Descripción de los Algoritmos

Los Algoritmos Meméticos (AMs) resultan de hibridar el AGG que mejor resultado haya proporcionado con una nueva BLS descrita en el Seminario 3. Se estudiarán las tres posibilidades de hibridación siguientes:

1. *AM-All*: Cada 10 generaciones, se aplica la BL sobre todos los cromosomas de la población.
2. *AM-Rand*: Cada 10 generaciones, se aplica la BL sobre un subconjunto de cromosomas de la población seleccionado aleatoriamente con probabilidad p_{LS} igual a 0,1 para cada cromosoma.
3. *AM-Best*: Cada 10 generaciones, aplicar la BL sobre los $0,1 \cdot N$ mejores cromosomas de la población actual (donde N es el tamaño de ésta).

5.3 Valores de los parámetros y ejecuciones

El tamaño de la población del AM será de 50 cromosomas. Las probabilidades de cruce y mutación serán 0,8 (por cromosoma) y 0,1/número de genes (por gen) en ambos casos. Cuando la BLS no produce cambios en el cromosoma decimos que falla. El contador de fallos está diseñado para evitar que se desperdicien evaluaciones de la función objetivo en cromosomas de mucha calidad. Permitiremos como mucho ϵ fallos en la BLS, siendo $\epsilon = 0,1 \cdot N$. El criterio de parada del AM consistirá en realizar 100000 evaluaciones de la función objetivo, incluidas por supuesto las de la BLS. Se aplicará la BL cada 10 generaciones del algoritmo, y cada vez que se aplique la BL se hará durante 100 evaluaciones (si no converge antes, claro).

Algorithm 1 Búsqueda Local Suave, BLS

```

1: function BLS( $S$ ,  $bestfit$ ,  $maxevals$ ,  $\epsilon$ )
2:    $RSI \leftarrow RandomShuffle(1, \dots, n)$ 
3:    $fallos \leftarrow 0$ 
4:    $i \leftarrow 0$ 
5:    $newsol \leftarrow S$ 
6:   while ( $fallos < \epsilon$ )  $\wedge$  ( $i < N$ )  $\wedge$  ( $evals < maxevals$ ) do
7:      $p \leftarrow RSI[i]$ 
8:      $oldvalue \leftarrow sol[p]$ 
9:     for  $val$  in  $1..k$  and  $evals < maxevals$  do
10:       $newsol[p] \leftarrow val$ 
11:       $fit \leftarrow fitness(newsol)$ 
12:       $evals \leftarrow evals + 1$ 
13:      if  $fit < bestfit$  then
14:         $S \leftarrow newsol$ 
15:         $bestfit \leftarrow fit$ 
16:      else
17:         $newsol[p] \leftarrow S[p]$ 
18:      end if
19:    end for
20:    if  $oldvalue = newsol[p]$  then
21:       $fallos \leftarrow fallos + 1$ 
22:    end if
23:     $i \leftarrow i + 1$ 
24:  end while
25:  return  $S$ ,  $bestfit$ ,  $evals$ 
26: end function

```

6 TABLAS DE RESULTADOS A OBTENER

Se mostrará una tabla global por cada problema y nivel de restricciones, en el que mostrará para cada algoritmo (COPKM, Random, BL, GG-UN, AGG-SF, AGE-UN, AGE-SF, AM-All, AM-Rand y AM-Best) los resultados de la ejecución de dicho algoritmo en los conjuntos de datos considerados. Para rellenar esta tabla se hará uso de los resultados medios mostrados en las tablas parciales. Aunque en la tabla que sirve de ejemplo se han incluido todos los algoritmos considerados en esta práctica, naturalmente sólo se incluirán los que se hayan desarrollado. Por tanto, se tendrán que hacer 6 tablas, que tendrán la misma estructura que las Tablas 1.

Tabla 1: Resultados obtenidos para el dataset XX con XX % de restricciones

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (seg)
Greedy	x	x	x	1	x
Random	x	x	x	x	x
BL	x	x	x	x	x
AGG-UN	x	x	x	x	x
AGG-SF	x	x	x	x	x
AGE-UN	x	x	x	x	x
AGE-SF	x	x	x	x	x
AM-All	x	x	x	x	x
AM-Rand	x	x	x	x	x
AM-Best	x	x	x	x	x

Adicionalmente, se deberá de realizar para cada ejemplo una visualización en boxplot mostrando la distribución de datos de cada algoritmo sobre la medida de fitness. Un ejemplo sería la gráfica de la Figura 1, ya incluida en la práctica anterior.

Nota importante: Dado que hay muchos algoritmos, es necesario en el análisis comparar los algoritmos que tengan mayor similitud para analizar la influencia del elemento que los diferencia, y justificar la diferencia en base a los conocimientos de los algoritmos.

A partir de los datos mostrados en estas tablas, el estudiante realizará un análisis de los resultados obtenidos, **que influirá significativamente en la calificación de la práctica**. En dicho análisis

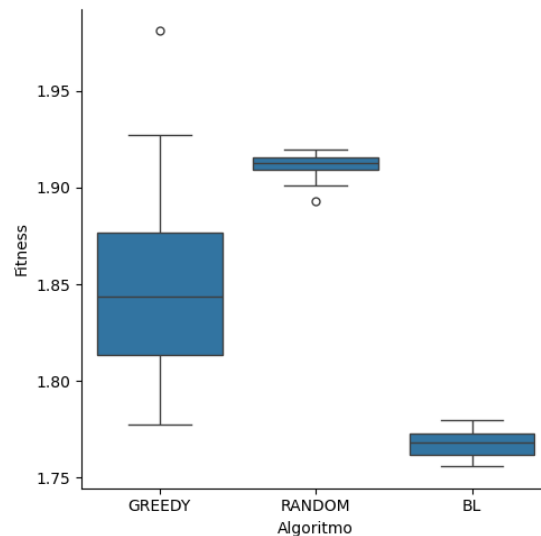


Figura 1: Ejemplo de Boxplot con los 100 valores por algoritmo

se deben comparar los distintos algoritmos en términos de calidad de las soluciones y tiempo requerido para producirlas. Por otro lado, se puede analizar también el comportamiento de los algoritmos en algunos de los casos individuales que presenten un comportamiento más destacado.

7 DOCUMENTACIÓN Y FICHEROS A ENTREGAR

En general, la **documentación** de ésta y de cualquier otra práctica será un fichero pdf que deberá incluir, al menos, el siguiente contenido:

- Portada con el número y título de la práctica (con el nombre del problema), el curso académico, el nombre, DNI y dirección e-mail del estudiante, y su horario de prácticas.
- Índice del contenido de la documentación con la numeración de las páginas.
- Breve** descripción/formulación del problema (**máximo 1 página**). Podrá incluirse el mismo contenido repetido en todas las prácticas presentadas por el estudiante.
- Breve descripción de la aplicación de los algoritmos empleados al problema (**máximo 4 páginas**): Todas las consideraciones comunes a los distintos algoritmos se describirán en este apartado, que será previo a la descripción de los algoritmos específicos. Incluirá por ejemplo la descripción del esquema de representación de soluciones y la **descripción en pseudocódigo (no código)** de la función objetivo y los operadores comunes, como los operadores de selección, mutación y los distintos operadores de cruce.
- Descripción en **pseudocódigo** de la **estructura del método de búsqueda** y de todas aquellas **operaciones relevantes** de cada algoritmo. Este contenido, específico a cada algoritmo se detallará en los correspondientes guiones de prácticas. El pseudocódigo **deberá forzosamente reflejar la implementación/el desarrollo realizados** y no ser una descripción genérica extraída de las transparencias de clase o de cualquier otra fuente. La descripción de cada algoritmo no deberá ocupar más de **2 páginas**.

Para esta primera práctica, se incluirán al menos las descripciones en pseudocódigo de:

- Para los AGs, el esquema de evolución y de reemplazamiento considerados.
 - Para los AMs, el esquema de búsqueda seguido por cada algoritmo en lo que
- Breve explicación de la **estructura del código de la práctica**, incluyendo un pequeño **manual de usuario** describiendo el proceso para que el profesor de prácticas pueda compilarlo (usando un sistema automático como **make** o similar) y cómo ejecutarlo, dando algún ejemplo de ejecución.
 - Experimentos y análisis de resultados:

- Descripción de los casos del problema empleados y de los valores de los parámetros considerados en las ejecuciones de cada algoritmo (**incluyendo las semillas utilizadas**).
 - Resultados obtenidos según el formato especificado.
 - Análisis de resultados. El análisis deberá estar orientado a **justificar** (según el comportamiento de cada algoritmo) **los resultados** obtenidos en lugar de realizar una mera “lectura” de las tablas. Se valorará la inclusión de otros elementos de comparación tales como gráficas de convergencia, boxplots, análisis comparativo de las soluciones obtenidas, representación gráfica de las soluciones, etc.
- h) Referencias bibliográficas u otro tipo de material distinto del proporcionado en la asignatura que se haya consultado para realizar la práctica (en caso de haberlo hecho).

Aunque lo esencial es el contenido, también debe cuidarse la presentación y la redacción. **La documentación nunca deberá incluir listado total o parcial del código fuente.**

En lo referente al **desarrollo de la práctica**, se entregará una carpeta llamada **software** que contenga una versión ejecutable de los programas desarrollados, así como el código fuente implementado o los ficheros de configuración del framework empleado. El código fuente o los ficheros de configuración se organizarán en la estructura de directorios que sea necesaria y deberán colgar del directorio *src* en el raíz. Junto con el código fuente, hay que incluir los ficheros necesarios para construir los ejecutables según el entorno de desarrollo empleado (tales como *.prj, makefile, *.ide, etc.). En este directorio se adjuntará también un pequeño fichero de texto de nombre LEEME que contendrá breves reseñas sobre cada fichero incluido en el directorio. Es importante que los programas realizados puedan leer los valores de los parámetros de los algoritmos desde fichero, es decir, que no tengan que ser recompilados para cambiar éstos ante una nueva ejecución. Por ejemplo, la semilla que inicializa la secuencia pseudoaleatoria debería poder especificarse como un parámetro más.

En el caso de que en el lenguaje de programación elegido se haya ofrecido un API, deberá de **obligatoriamente implementarlo siguiendo el API ofrecido**. Si no fuese el caso, se adaptará el API recomendado.

El fichero pdf de la documentación y la carpeta software serán comprimidos en un fichero .zip etiquetado con los apellidos y nombre del estudiante (Ej. Pérez Pérez Manuel.zip). Este fichero será entregado por internet a través del espacio de la asignatura en PRADO.

8 MÉTODO DE EVALUACIÓN

Al principio de la práctica se ha indicado la puntuación máxima que se puede obtener por cada algoritmo y su análisis. **La inclusión de trabajo voluntario** (desarrollo de variantes adicionales, experimentación con diferentes parámetros, prueba con otros operadores o versiones adicionales del algoritmo, análisis extendido, etc.) **podrá incrementar la nota final** por encima de la puntuación máxima definida inicialmente, o compensar parcialmente errores en la práctica.

En caso de que el comportamiento del algoritmo en la versión implementada/ desarrollada no coincida con la descripción en pseudocódigo o no incorpore las componentes requeridas, se podría reducir hasta en un 50 % la calificación del algoritmo correspondiente.